MILE STONE – 4 : DEPLOYMENT

DEPLOYMENT URL LINK

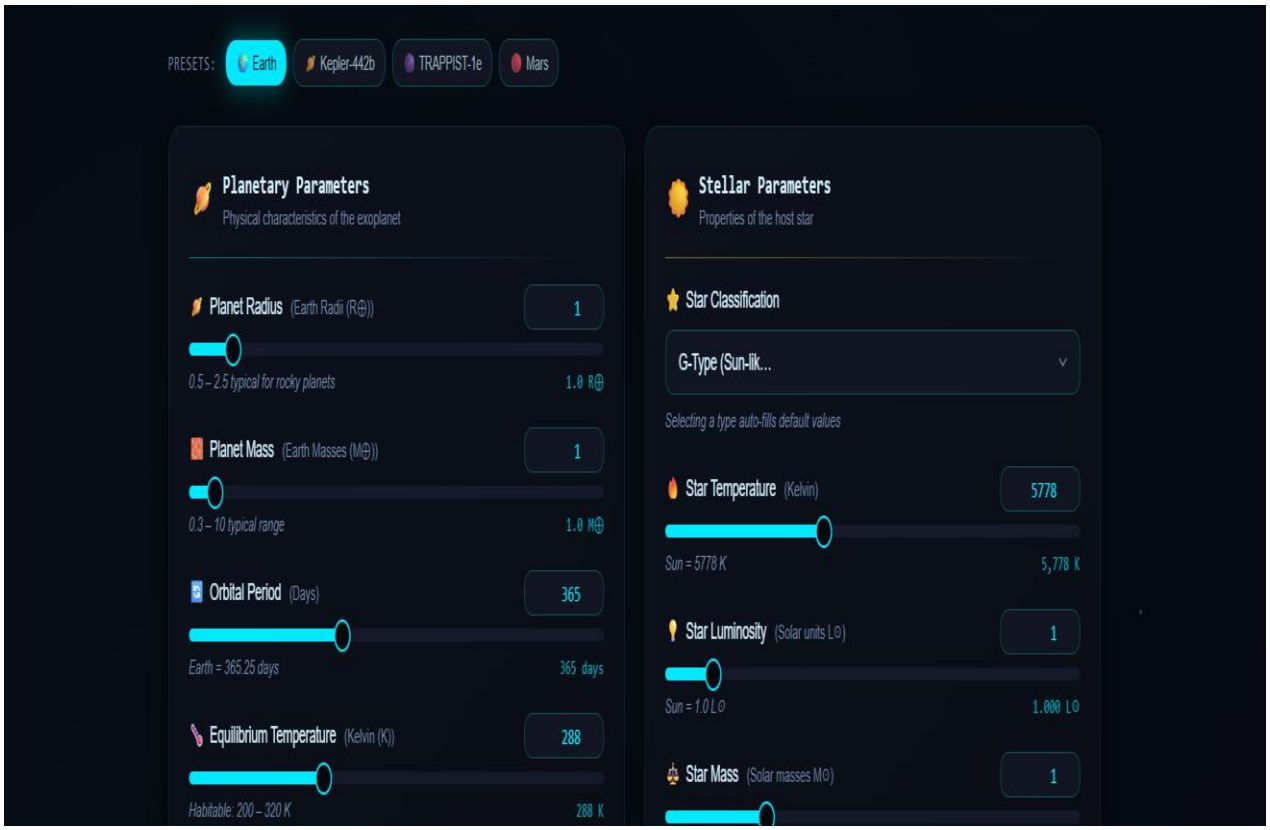
FRONTEND – https://exohabitai-1-codx.onrender.com/

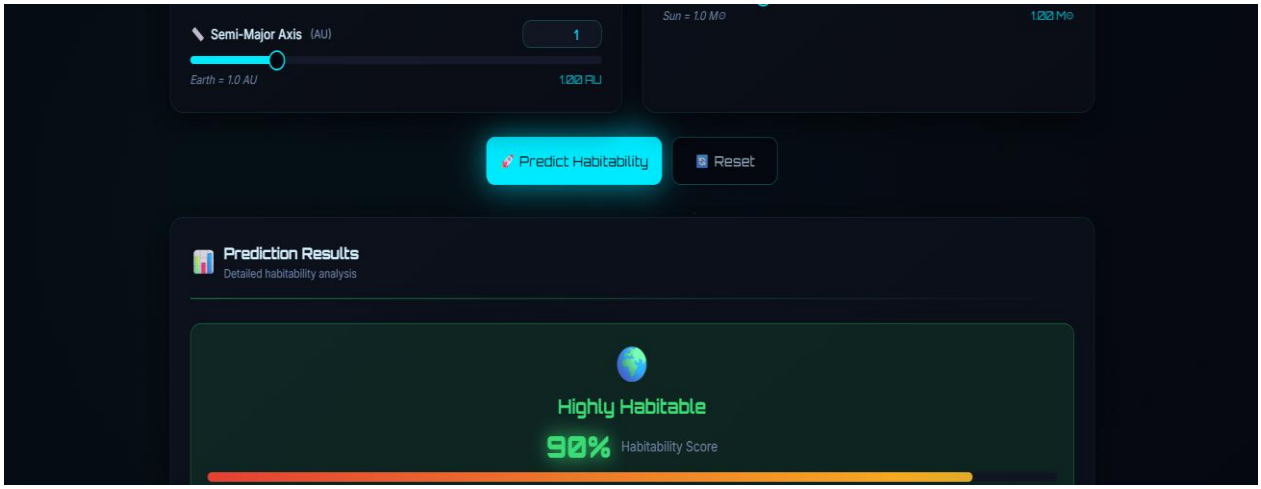
BACKEND – https://exohabitai-ux25.onrender.com

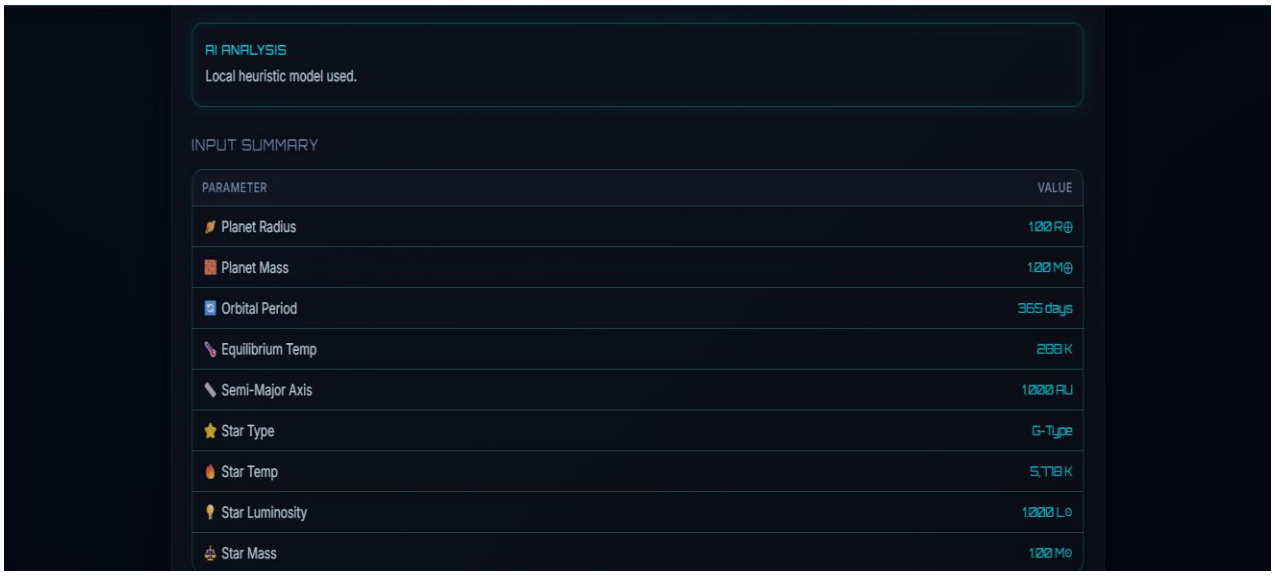
GITHUB REPOSITORY LINK –

https://github.com/GKSJ19/ExoHabitAI/tree/B.V.Harini

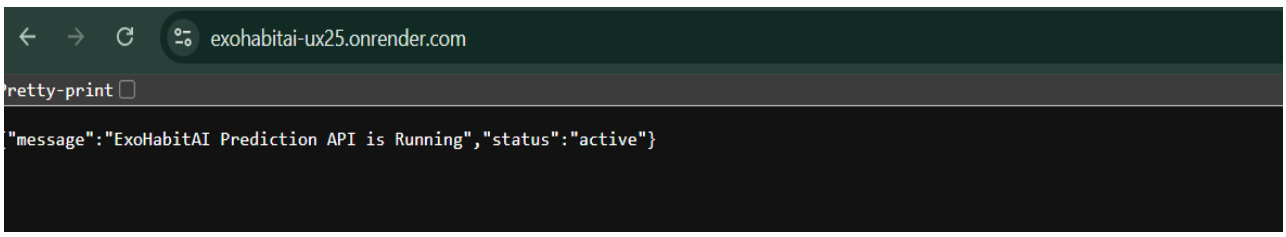
SCREENSHOT OF WORKING WEB




Semi-Major Axis (AU)
1
Earth = 1.0 AU
100 AU
Sun = 1.0 M⊙
100 M⊙
Predict Habitability
Reset
Prediction Results
Detailed habitability analysis
Highly Habitable
90%
Habitability Score


AI ANALYSIS
Local heuristic model used.
INPUT SUMMARY
PARAMETER
VALUE
Planet Radius
100 R⊕
Planet Mass
100 M⊕
Orbital Period
365 days
Equilibrium Temp
288 K
Semi-Major Axis
1000 AU
Star Type
G-Type
Star Temp
5,778 K
Star Luminosity
1000 L⊙
Star Mass
100 M⊙


Star Mass
100 M⊙
Planet Rankings
1 planet analyzed
CLEAR ALL
Earth
90%
Highly Habitable
11:09:36 PM


exohabitai-ux25.onrender.com
Pretty-print
{"message":"ExoHabitAI Prediction API is Running","status":"active"}

API FUNCTIONALITY CONFIRMATION

Health Check (GET /status): Returns a successful JSON response confirming the Random Forest model is loaded and operational.

Prediction Engine (POST /predict): Successfully accepts 39-parameter JSON payloads, validates the input structure, processes the data through the Stacking Ensemble, and returns accurate habitability classifications.

CORS Configuration: Cross-Origin Resource Sharing (CORS) has been securely configured in app.py to allow the Vercel frontend to fetch data without browser security blocks.

DEPLOYMENT ARCHITECTURE

● Backend: Flask & Machine Learning Model hosted on Render (Web Service).

● Frontend: React & Vite UI hosted on Vercel .

BACKEND DEPLOYMENT

(Render) The backend serves the Random Forest stacking model and exposes the REST API endpoints.

Prerequisites Configuration:

● app.py: Contains the main Flask application and routes.

● requirements.txt: Includes flask, flask-cors, gunicorn, joblib, scikit-learn, xgboost, pandas, numpy.

● Procfile: Contains web: gunicorn backend.app:app to define the WSGI server.

Render Setup Steps:

1. Navigated to Render Dashboard → New + → Web Service.

2. Connected the GitHub repository.

3. Applied the following configuration:

- Language/Runtime: Python 3
- Root Directory: backend (Critical: Instructs Render to execute within the backend folder)
- Build Command: pip install -r requirements.txt
- Start Command: gunicorn app:app

4. Deployed on the Free Tier to generate the live API URL.

Frontend Deployment (Vercel)

The frontend provides the interactive 3D UI and securely connects to the Render API.

Prerequisites Configuration:

● The API base URL was made dynamic in

frontend\habitable-worlds-finder-main\src\lib\habitability.ts

```
● JavaScript const API_URL = import.meta.env.VITE_API_URL ||
"http://127.0.0.1:5000";
```

Vercel Setup Steps:

1. Imported the GitHub repository into Vercel as a new Project.

2. Configured the project settings:

   - Framework Preset : Vite
   - Root Directory: frontend/habitable-worlds-finder-main

3. Executed deployment to generate the initial Vercel production URL.